CampusFind

College Lost and Found Portal

Project Report and Technical Documentation

Submitted by: Albin Jakson

Register/Roll Number: 225707

Department: B tech CSE

Institution: sahrdaya college of engineering and technology thrissur

Academic Year: 2026 - 27

Project Guide: Dr.Anil Antony

Declaration

I declare that this report describes the CampusFind project prepared as part of my academic work. Complete any institution-specific declaration and signature requirements before submission.

Acknowledgement

I thank my project guide, department, institution, and the maintainers of the technologies used for their guidance and support.

Table of Contents

Abstract

1. Introduction
2. Problem Statement
3. Objectives
4. Scope
5. Existing and Proposed System
6. Requirements
7. Technology Stack

8. System Architecture

9. Design and Implementation

10. API Documentation

11. Environment Configuration

12. Deployment

13. Testing and Results

14. Security Considerations

15. Limitations and Future Enhancements

16. Conclusion

17. References

Appendix A: Local Setup

Appendix B: Troubleshooting

Abstract

CampusFind is a web-based college lost-and-found portal intended to centralize reports about lost and found belongings. The application uses React with Vite for the frontend, Node.js and Express for the backend API, and MongoDB through Mongoose for data persistence. The frontend is deployed on Vercel, the backend on Render, and the database is hosted through MongoDB Atlas. The working workflow allows users to view existing reports and submit a report. The deployed API health and item endpoints were tested. A production CORS issue caused by a trailing slash in CLIENT_ORIGIN was resolved by making the configured origin exactly match the Vercel site origin.

1. Introduction

Lost belongings are common in educational institutions. Information shared through word of mouth, notice boards, or separate messaging groups can be difficult to discover and maintain. CampusFind provides a central web interface for viewing and submitting lost-and-found reports. The project demonstrates frontend development, API design, database connectivity, environment configuration, and cloud deployment.

2. Problem Statement

A college community may not have a single, searchable location to publish lost-and-found information. Scattered reports can be missed or become outdated. CampusFind addresses this problem by providing a centralized website backed by a database.

3. Objectives

Provide a central interface for lost-and-found reports.

Allow users to submit reports through a form.

Store and retrieve reports using MongoDB.

Separate frontend and backend responsibilities.

Deploy the application online.

Configure production API access and CORS correctly.

Provide a foundation for future search, authentication, and administrative functionality.

4. Scope

Confirmed implemented/tested: React/Vite frontend; Express API on Node.js; MongoDB connection through Mongoose; GET /api/health; item routes under /api/items; viewing existing reports; submitting a report in local testing; Vercel frontend deployment; Render backend deployment; MongoDB Atlas hosting; production CORS correction. Features such as login, image upload, notifications, and admin workflows should only be described as implemented if they are present in the current source code.

5. Existing and Proposed System

An existing manual process may rely on verbal communication, physical notice boards, or multiple chat groups. The proposed CampusFind system centralizes reports in a web application. The browser requests data from the API, the API communicates with MongoDB, and returned JSON is displayed in the frontend.

6. Requirements

Functional: load the report interface; retrieve reports from the backend; display reports; submit a report; expose a health endpoint; connect to MongoDB when configured.

Non-functional: usability, maintainability, availability, persistent storage, browser compatibility, and protection of secrets.

Software: Node.js/npm, a modern browser, Visual Studio Code or another editor, Git/GitHub, MongoDB Atlas, Vercel, and Render.

7. Technology Stack

React provides the component-based interface. Vite supports local development and production builds. Node.js runs the backend, and Express provides HTTP routing and middleware. MongoDB stores report documents, while Mongoose manages the database connection and

models. dotenv loads environment variables. Git/GitHub stores source code. Vercel hosts the frontend and Render hosts the backend API.

8. System Architecture

The application has three logical layers: (1) presentation layer—React frontend; (2) application layer—Express API on Node.js; and (3) data layer—MongoDB accessed through Mongoose. Request flow: the user opens the Vercel site; React calls the configured API URL; the browser sends a request to Render; Express processes the route; Mongoose reads or writes MongoDB; JSON is returned and rendered by React.

9. Design and Implementation

The frontend uses React state/effect logic to load reports and handle submission. API calls are organized in client/src/services/api.js, including getItems and createItem. The backend entry point is server/src/server.js. It loads environment variables, configures CORS, JSON parsing and request logging, exposes /api/health, mounts item routes at /api/items, and installs not-found/error middleware. The database configuration reads MONGODB_URI and connects through Mongoose. Verify the current item model and route files before documenting exact field names, validation rules, or response shapes.

10. API Documentation

Base URL (production): https://campusfind-lost-and-found.onrender.com

GET /api/health — checks whether the API is responding. Example response: {"status":"ok","service":"College Lost and Found API"}.

GET /api/items — retrieves item reports as JSON. The precise response shape depends on the current route implementation.

POST /api/items — creates a report. Send application/json using the exact fields expected by the current route/model validation. Confirm required fields and status codes in the source before publishing a formal API contract.

11. Environment Configuration

Backend server/.env for local development:
PORT=5000
MONGODB_URI=your_mongodb_atlas_connection_string
CLIENT_ORIGIN=http://localhost:5173

Frontend client/.env for local development:
VITE_API_URL=http://localhost:5000/api

Vercel production VITE_API_URL:
https://campusfind-lost-and-found.onrender.com/api

Render CLIENT_ORIGIN:
https://campusfind-lost-and-found-xi.vercel.app

The production CLIENT_ORIGIN must match the browser origin exactly and must not include a trailing slash. Never commit real .env files or credentials.

12. Deployment

Source is maintained in GitHub. The frontend is deployed to Vercel and the backend API to Render; MongoDB Atlas provides hosted database storage. In Vercel, configure VITE_API_URL and redeploy after changing it because Vite reads environment variables during build. In Render, configure MONGODB_URI and CLIENT_ORIGIN, then redeploy after changes. Test /api/health and /api/items, then verify report creation and persistence on the production site.

13. Testing and Results

Observed tests: local frontend opened successfully; local backend connected to MongoDB; local item endpoint returned report data; production /api/health returned status ok; production /api/items returned JSON report data or an empty array. The deployed frontend initially could not fetch reports because CLIENT_ORIGIN ended with a slash while the browser origin did not. Removing the trailing slash and redeploying fixed the CORS issue. Retest production report submission and persistence before marking those production tests as passed.

14. Security Considerations

Keep .env files out of GitHub. Rotate any database password that may have been exposed and update Render's MONGODB_URI. Use appropriately limited database permissions and intentional Atlas network-access rules. Validate user input on the server, avoid collecting unnecessary personal information, and consider rate limiting and access controls before wider public use. VITE_API_URL is public frontend configuration and must never contain credentials.

15. Limitations and Future Enhancements

Possible future work includes authentication, role-based administration, report moderation, search and filtering, image upload, claim verification, notifications, pagination, automated tests, accessibility improvements, and a privacy/retention policy. These are proposed enhancements unless verified in the current source code.

16. Conclusion

CampusFind demonstrates a full-stack web application for college lost-and-found reports. React/Vite provides the interface, Express/Node.js provides the API, and MongoDB/Mongoose

provides persistence. The application is deployed using Vercel and Render with MongoDB Atlas. The production health and items endpoints were tested, and the CORS issue was resolved by removing the trailing slash from CLIENT_ORIGIN. Future development can strengthen security and improve discovery and administration features.

17. References

React: https://react.dev/
Vite: https://vite.dev/
Node.js: https://nodejs.org/docs/
Express: https://expressjs.com/
MongoDB: https://www.mongodb.com/docs/
Mongoose: https://mongoosejs.com/docs/
Vercel: https://vercel.com/docs
Render: https://render.com/docs
GitHub: https://docs.github.com/

Appendix A: Local Setup

1. Install Node.js and npm.
2. Open the project in Visual Studio Code.
3. In server, install dependencies with npm install and configure server/.env.
4. In client, install dependencies and configure client/.env with VITE_API_URL=http://localhost:5000/api.
5. Start each application using scripts listed in the respective package.json files. The backend script used in this project is npm run dev; the frontend commonly uses npm run dev.
6. Open the Vite URL printed in the terminal, usually http://localhost:5173.

On Windows PowerShell, if npm.ps1 is blocked, use npm.cmd instead of changing execution policy.

Appendix B: Troubleshooting

Frontend cannot connect: check the backend health endpoint, verify VITE_API_URL points to the production API, then redeploy Vercel.

CORS error: ensure CLIENT_ORIGIN exactly matches the deployed Vercel origin with no trailing slash; redeploy Render.

Database failure: verify MONGODB_URI, Atlas database-user credentials, and network access settings. Never share credentials in public messages.

Reports missing: inspect browser Console/Network tabs, the API response, Render logs, and route/model validation.